LONDON METROPOLITAN UNIVERSITY

ITAHARI INTERNATIONAL COLLEGE

Module Code & Module Title:

CS4051NT Fundamentals of Computing

Assessment Weightage & Type:

70% Individual Coursework

Year and Semester:

2025 Spring, 2nd Semester

Student Name: Sugam Khatiwada

London Met ID: 24046096

College ID: NP05CP4A240169

Assignment Due Date: May 14, 2025

Assignment Submission Date: May 14, 2025

Submitted To: Ajayraj Bhattarai

Word Count: 4927

I confirm that I understand my coursework needs to be submitted online via MySecondTeacher under the relevant module page before the deadline in order for my assignment to be accepted and marked. I am fully aware that late submissions will be treated as non-submission and a mark of zero will be awarded.

24046096 Sugam Khatiwada

Islington College,Nepal

Document Details

Submission ID
trn:oid:::3618:95317047

Submission Date
May 11, 2025, 3:51 PM GMT+5:45

Download Date
May 11, 2025, 4:00 PM GMT+5:45

File Name
24046096 Sugam Khatiwada

File Size
32.8 KB

46 Pages

4,927 Words

25,217 Characters

turnitin Page 1 of 51 - Cover Page Submission ID trn:oid:::3618:95317047

9% Overall Similarity

The combined total of all matches, including overlapping sources, for each database.

Match Groups

- **49** Not Cited or Quoted 9%
  Matches with neither in-text citation nor quotation marks
- **0** Missing Quotations 0%
  Matches that are still very similar to source material
- **1** Missing Citation 0%
  Matches that have quotation marks, but no in-text citation
- **0** Cited and Quoted 0%
  Matches with in-text citation present, but no quotation marks

Top Sources

2% Internet sources
0% Publications
9% Submitted works (Student Papers)

Integrity Flags

0 Integrity Flags for Review

Our system's algorithms look deeply at a document for any inconsistencies that would set it apart from a normal submission. If we notice something strange, we flag it for you to review.

A Flag is not necessarily an indicator of a problem. However, we'd recommend you focus your attention there for further review.

Match Groups

- **49** Not Cited or Quoted 9%
  Matches with neither in-text citation nor quotation marks
- **0** Missing Quotations 0%
  Matches that are still very similar to source material
- **1** Missing Citation 0%
  Matches that have quotation marks, but no in-text citation
- **0** Cited and Quoted 0%
  Matches with in-text citation present, but no quotation marks

Top Sources

- 2% Internet sources
- 0% Publications
- 9% Submitted works (Student Papers)

Top Sources

The sources with the highest number of matches within the submission. Overlapping sources will not be displayed.

1	Submitted works	Asia Pacific University College of Technology and Innovation (UCTI) on 2021-05-20 <1%
2	Submitted works	Asia Pacific University College of Technology and Innovation (UCTI) on 2024-08-16 <1%
3	Submitted works	Lincoln University on 2024-03-26 <1%
4	Internet	www.coursehero.com <1%
5	Submitted works	Asia Pacific University College of Technology and Innovation (UCTI) on 2021-11-14 <1%
6	Submitted works	The British College on 2019-05-21 <1%
7	Submitted works	Burnley College, Lancashire on 2025-01-20 <1%
8	Submitted works	NCC Education on 2024-02-23 <1%
9	Submitted works	Eynesbury Institute of Business and Technology on 2023-04-10 <1%
10	Submitted works	University of Maryland, Global Campus on 2024-07-31 <1%

11 Submitted works
University of Ulster on 2025-05-07 <1%

12 Submitted works
islingtoncollege on 2025-05-06 <1%

13 Submitted works
The Robert Gordon University on 2025-04-20 <1%

14 Submitted works
The University of Wolverhampton on 2022-04-27 <1%

15 Submitted works
AlHussein Technical University on 2023-02-03 <1%

16 Submitted works
Polytechnic Institute Australia on 2023-09-24 <1%

17 Submitted works
Central Queensland University on 2023-08-25 <1%

18 Internet
ebin.pub <1%

19 Internet
www.jvejournals.com <1%

20 Submitted works
Asia Pacific University College of Technology and Innovation (UCTI) on 2023-06-02 <1%

21 Submitted works
60892 on 2015-06-15 <1%

22 Submitted works
Asia Pacific University College of Technology and Innovation (UCTI) on 2023-07-06 <1%

23 Submitted works
Nottingham Trent University on 2024-08-15 <1%

24 Submitted works
University of Bolton on 2025-01-08 <1%

25 Submitted works
Icon College of Technology and Management on 2018-06-23 <1%

26 Submitted works
TAFE Queensland Brisbane on 2024-05-11 <1%

27 Internet
www.toptal.com <1%

28 Submitted works
International School of Management and Technology on 2018-06-10 <1%

29 Submitted works
Liverpool John Moores University on 2020-01-05 <1%

30 Submitted works
Melbourne Institute of Technology on 2018-09-28 <1%

31 Submitted works
Nottingham Trent University on 2013-05-07 <1%

32 Submitted works
Queen's University of Belfast on 2024-12-04 <1%

33 Submitted works
University Tun Hussein Onn Malaysia on 2020-07-19 <1%

34 Submitted works
University of Lugano on 2010-07-22 <1%

35 Submitted works
islingtoncollege on 2025-05-07 <1%

36 Submitted works
Informatics Education Limited on 2012-01-01 <1%

37 Submitted works
Asia Pacific University College of Technology and Innovation (UCTI) on 2023-08-14 <1%

38 Submitted works
Xiamen University on 2023-07-31 <1%

Table of Contents

Table of Figures

Table of Tables

1 Introduction

1.1 Project Overview

This project is made for the local skincare and beauty product business called WeCare. The goal of this project is to help the business to maintain their inventory, handle the sales, and entirely automate the system for generating invoices for the customers and restocking records.

This project is entirely built using Python programming language. The product details are maintained in the .txt files, which are later stored in collection datatypes (lists, dictionaries, etc.) of Python and displayed to the customer in a clear and readable format.

It also has the feature of freebies (like “Buy Three, Get One Free”) and has the feature to show the selling price of the product increased by 200% of the marked price to the customers which is automated by the program itself.

1.2 Aims and Objectives

1.2.1 Aims

The aim of the project is to develop a python-based application for local vendor WeCare to help manage the inventory, handle the sales, improve customer experience and apply freebie policy.

1.2.2 Objectives

- To design fully functional application to sale skincare products.
- To read product details for the text file and store it in appropriate collection datatype.
- To display information of the products in the Python shell.
- To generate the invoices for sale and product restocking.
- To automatically update the stocks in the text file.

1.3 Python Programming Language

Python is a powerful high-level language known for its flexibility and simplicity. It is used for making different type of mobile applications, websites, desktop software and many more. One of the strong aspects of Python is that it supports multiple programming paradigms, which is basically the different style for writing the code (Aezion, 2019).

Figure 1 Python

Python supports three types of paradigms:

- Procedural Programming
- Functional Programming
- Object Oriented Programming (OOP)

1.3.1 Procedural Programming

It is simple and most commonly used paradigm. In procedural programming we write code step by step using variables, loops etc.

```
product = "Vitamin C Serum"
brand = "Garnier"
price = 1000

print("Product:", product)
print("Brand:", brand)
print("Price: Rs.", price)
```

Figure 2 Procedural Programming

1.3.2 Functional Programming

It focusses on writing code in reusable functions that takes input and return output without changing the other program.

```
def display_product(name, brand, price):
    print("Product:", name)
    print("Brand:", brand)
    print("Price: Rs.", price)

display_product("Vitamin C Serum", "Garnier", 1000)
```

Figure 3 Functional Programming

1.3.3 Object Oriented Programming (OOP)

It uses the concept of class and objects. Every object holds data and methods.

```
class Product:
    def __init__(self, name, brand, price):
        self.name = name
        self.brand = brand
        self.price = price

    def display_info(self):
        print("Product:", self.name)
        print("Brand:", self.brand)
        print("Price: Rs.", self.price)

item = Product("Vitamin C Serum", "Garnier", 1000)
item.display_info()
```

Figure 4 Object Oriented Programming

1.4 Tool Used

1.4.1 IDLE

IDLE (Integrated Development and Learning Environment) is one of the best code editor for Python, which comes pre-installed with Python. It is very beginner friendly due to its high-speed output and beginner friendly user interface. It makes debugging easy with clear messages (Python Software Foundation, n.d.). I used IDLE as my frontline code editor for this project.

Figure 5 IDLE

1.4.2 Draw.io

For creating flowchart for this project, I used Draw.io (also known as diagrams.net). It is free and easy to use diagramming software that allows users to create flowchart, class diagrams and other visual representation easily. Using this I was able to visually map out the entire process of my project. This tool provided various shapes like process box, decision diamond, arrows etc. to organize the logic of the program in understandable and clean format.

Figure 6 Draw.io

2 Algorithm

Algorithm is step by step procedure made to solve the problem or perform a specific task. It is the foundation for programming and computer science, which helps in ensuring accuracy, efficiency and problem solving. Here is the algorithm I created for my project:

Step 1: Start

Step 2: Read product file and store in product list

Step 3: Display menu

1. Display Product
2. Add Product
3. Sell Product
4. Exit

Step 4: Ask user to select one from the option

Step 5: If option 1 is selected

Step 5.1: Display the available products

Step 5.2: Go to step 3

Step 6: If option 2 is selected

Step 6.1: Ask the name of the product to add

Step 6.2: Check for the validity of name

Step 6.2.1: If name is not valid

Step 6.2.1.1: Ask for the correct name and go to step 6.2

Step 6.3: Check whether the product exists in the inventory

Step 6.3.1: If product exists

Step 6.3.1.1: Ask for the quantity to add

Step 6.3.1.2: Check the entered quantity validity

Step 6.3.1.2.1: If quantity is not valid

Step 6.3.1.2.1.1: Ask for the correct quantity and go to step 6.3.1.2

Step 6.3.1.3: Update the quantity in product list and shoppingcart list

Step 6.3.2: If product doesn't exist

Step 6.3.2.1: Ask for the brand name

Step 6.3.2.2: Check the validity of the entered brand name

Step 6.3.2.2.1: If brand name is not valid

Step 6.3.2.2.1.1: Ask for the correct brand name and go to step 6.3.2.2

Step 6.3.2.3: Ask for the price

Step 6.3.2.4: Check the validity of the price

Step 6.3.2.4.1: If price is not valid

Step 6.3.2.4.1.1: Ask for the valid price and go to step 6.3.2.4

Step 6.3.2.5: Ask for the quantity

Step 6.3.2.6: Check the validity of the quantity

Step 6.3.2.6.1: If quantity is not valid

Step 6.3.2.6.1.1: Ask for the valid quantity and go to step 6.3.2.6

Step 6.3.2.7: Ask for the country name

Step 6.3.2.8: Check the validity of the country name

Step 6.3.2.8.1: If name of the country not valid

Step 6.3.2.8.1.1: Ask for the valid country name and go to step 6.3.2.8

Step 6.3.2.9: Update the quantity in product list and shoppingcart list

Step 6.4: Ask whether to add more product or not

Step 6.4.1: If yes

Step 6.4.1.1: Go to step 6.1

Step 6.4.2: If no

Step 6.4.2.1: Update the product file

Step 6.4.2.2: Display the summary of the added product

Step 6.4.2.3: Ask for the vendor's name

Step 6.4.2.4: Check the validity of vendor name

Step 6.4.2.4.1: If vendor name not valid

Step 6.4.2.4.1.1: Ask for the valid vendor's name and go to step 6.4.2.4

Step 6.4.2.5: Create invoice in add_invoices folder

Step 6.4.2.6: Go to step 3

Step 7: If option 3 is selected

Step 7.1: Display the available products

Step 7.2: Ask for the product I'd like to buy.

Step 7.3: Check the validity of id entered

Step 7.3.1: If id not valid

Step 7.3.1.1: Ask to enter the valid id and go to step 7.3

Step 7.4: Check whether entered exist in the inventory

Step 7.4.1: If not exist

Step 7.4.1.1: Ask to enter id displayed in the available product

Step 7.5: Ask for the quantity of the product

Step 7.6: Check the validity of quantity entered

Step 7.6.1: If quantity entered is not valid

Step 7.6.1.1: Ask for valid quantity and go to step 7.6

Step 7.7: Calculate for the free item

Step 7.8: Update the product in product list, cart list and free list

Step 7.9: Ask whether to sell more product or not

Step 7.9.1: If yes

Step 7.9.1.1: Go to Step 7.2

Step 7.9.2: If no

Step 7.9.2.1: Update the product file

Step 7.9.2.2: Display the summary of sold product

Step 7.9.2.3: Ask for the customer's name

Step 7.9.2.4: Check the validity of customer name

Step 7.9.2.4.1: If name not valid

Step 7.9.2.4.1.1: Ask for the valid name and go to step 7.9.2.4

Step 7.9.2.5: Create invoices in sell_invoices folder

Step 7.9.2.6: Go to step 3

Step 8: If option other than 1,2,3,4 is selected

Step 8.1: Display enter from option 1,2,3,4

Step 8.2: Go to step 3

Step 9: If option 4 is selected

Step 9.1: Display exited the program and go to step 10

Step 10: End

3 Pseudocode

It is the way of writing the logic of the program in plain English text instead of actual programming syntax (Grishina, 2023). It helps to plan how a program will work before writing the actual code. Pseudocode does not follow any strict rule like Python does, it only focuses on describing the code clearly so that anyone understand it easily.

In this WeCare project, I used pseudocode to break down each module into simple steps. This helped me to understand logic easily and made easier to implement in python later.

3.1 Read.py Module

FUNCTION load_products():

INITIALIZE empty products list

TRY

OPEN "Product.txt" in read mode as file

READ all lines from file

FOR each line in lines:

REMOVE whitespace with strip()

SPLIT line by comma and space

ADD resulting product list to products

RETURN products list

CATCH Exception as e

PRINT error message with exception details

RETURN empty list

3.2 Write.py Module

IMPORT datetime module

FUNCTION save_products(products):

TRY

OPEN "Product.txt" in write mode as file

FOR each product in products:

JOIN product elements with comma and space

WRITE joined string to file with newline

CATCH Exception as e

PRINT error message with exception details

FUNCTION add_invoices(shopping_cart, vendor):

GENERATE invoice_name using current timestamp

TRY

OPEN file at "Add_Invoices/{vendor}_{invoice_name}.txt" in write mode

WRITE invoice header with separators

WRITE centered invoice title

WRITE company name

WRITE vendor name

WRITE purchase date/time

WRITE column headers

INITIALIZE total_amount to 0

FOR each cart item in shopping_cart:

EXTRACT id, name, brand, price, quantity, country

TRUNCATE text fields if too long

WRITE formatted row with item details

ADD price × quantity to total_amount

WRITE table footer

WRITE total amount row

WRITE vat row

WRITE Grand Total row

WRITE closing line

PRINT success message

CATCH Exception as e

PRINT error message with exception details

FUNCTION sell_invoices(shopping_cart, customer, free):

GENERATE invoice_name using current timestamp

REPLACE spaces with underscores

REPLACE colons with hyphens

TRY

OPEN file at "Sell_Invoices/{customer}_{invoice_name}.txt" in write mode

WRITE invoice header with separators

WRITE centered invoice title

WRITE company name

WRITE customer name

WRITE purchase date/time

WRITE column headers including free item columns

INITIALIZE total_amount to 0

FOR each index i and cart item in shopping_cart:

EXTRACT id, name, brand, price, quantity, country

GET free_item count from free list at index i

TRUNCATE text fields if too long

WRITE formatted row with item details including free items

ADD price × (quantity - free_item) to total_amount

WRITE table footer

WRITE total amount row

WRITE vat row

WRITE Grand Total row

WRITE closing line

PRINT success message

CATCH Exception as e

PRINT error message with exception details

3.3 Operation.py module

IMPORT write module

FUNCTION display_products(products):

PRINT table header with properly formatted columns for ID, Name, Brand, Price, Quantity, Country

FOR EACH product in products:

EXTRACT id, name, brand, price, quantity, country from product

TRUNCATE name, brand, country if too long

SET price to double the integer value of product[3]

SET quantity to product[4] multiplied by 0.75

PRINT formatted row with product details

PRINT table footer

FUNCTION add_summary(shopping_cart):

PRINT checkout summary header and column headers

INITIALIZE total to 0

FOR EACH cart item in shopping_cart:

EXTRACT name (truncated if needed), price and quantity

PRINT formatted row with item details

ADD price × quantity to total

PRINT table footer

PRINT total amount

PRINT closing line

FUNCTION sell_summary(shopping_cart, free):

PRINT checkout summary header and column headers

INITIALIZE total to 0

FOR EACH index i and cart item in shopping_cart:

GET free_item count from free list

EXTRACT name (truncated if needed), price and quantity

PRINT formatted row with item details including free items

ADD price × (quantity - free items) to total

PRINT table footer

PRINT total amount

PRINT closing line

FUNCTION add_product(products):

INITIALIZE empty shopping_cart list

WHILE True:

SET exist flag to False

REPEAT

GET product name input

CONVERT name to uppercase

VALIDATE name is not empty

UNTIL valid name is entered

FOR EACH product in products:

IF product name matches input name:

REPEAT

GET quantity to add

VALIDATE quantity is positive integer

UNTIL valid quantity is entered

UPDATE product quantity

SET exist flag to True

CREATE cart item with product details

ADD to shopping_cart

BREAK LOOP

IF exist is False:

GENERATE new ID based on product list length

REPEAT

GET brand name input

CONVERT brand to uppercase

VALIDATE brand is not empty

UNTIL valid brand is entered

REPEAT

GET price input

VALIDATE price is positive integer

UNTIL valid price is entered

REPEAT

GET quantity input

VALIDATE quantity is positive integer

UNTIL valid quantity is entered

REPEAT

GET country input

CONVERT country to uppercase

VALIDATE country is not empty and contains only letters or spaces

UNTIL valid country is entered

ADD new product to products list

CREATE cart item with product details

ADD to shopping_cart

REPEAT

ASK if user wants to add another product

GET input and convert to uppercase

UNTIL user enters Y or N

IF input is N:

BREAK loop

SAVE updated products using write.save_products()

CALL add_summary() to display purchase summary

REPEAT

GET vendor name input

CONVERT vendor to uppercase

VALIDATE vendor is not empty and contains only letters or spaces

UNTIL valid vendor name is entered

GENERATE invoice with write.add_invoices()

PRINT success message

```
    RETURN updated products


FUNCTION sell_product(products):
    INITIALIZE empty shopping_cart and free lists
    DISPLAY all products

    WHILE True:
        GET product ID input
        SET exist flag to False
        SET quantitynotFound flag to False

        FOR EACH product in products:
            IF product ID matches input ID:
                SET exist flag to True

                REPEAT
                    GET quantity to buy
                    CALCULATE free items (quantity divided by 3)
                    CALCULATE available quantity (product[4] multiplied by 0.75)

                    IF quantity is negative:
                        PRINT error message
                    ELSE IF stock is zero:
```

SET quantity to 0

SET free_item to 0

PRINT out of stock message

SET quantitynotFound to True

BREAK LOOP

ELSE IF available quantity is less than requested quantity:

PRINT insufficient stock message

ELSE IF available quantity is sufficient:

BREAK LOOP

UNTIL valid quantity is entered

UPDATE product quantity

CREATE cart item with product details

ADD free_item count to free list

ADD cart item to shopping_cart

BREAK LOOP

IF exist is False:

PRINT error message

CONTINUE LOOP

IF quantitynotFound is True **OR** quantitynotFound is False:

REPEAT

```
            ASK if user wants to buy another product
            GET input and convert to uppercase
        UNTIL user enters Y or N

        IF input is N:
            BREAK LOOP

SET i to 0
WHILE i < length of shopping_cart:
    IF shopping_cart[i][4] equals '0':
        REMOVE item at position i from shopping_cart
        REMOVE item at position i from free list
    ELSE:
        INCREMENT i

IF shopping_cart is empty:
    PRINT no products bought message
    RETURN products
ELSE:
    SAVE updated products using write.save_products()
    CALL sell_summary() to display purchase summary with free items

    REPEAT
```

GET customer name input

CONVERT customer to uppercase

VALIDATE customer is not empty and contains only letters or spaces

UNTIL valid customer name is entered

GENERATE invoice with write.sell_invoices()

PRINT success message

RETURN updated products

3.4 Main.py Module

IMPORT read

IMPORT write

IMPORT operation

FUNCTION menu()

  DISPLAY formatted menu box with options 1-4

  TRY

    GET user choice as integer

  EXCEPT value error

    DISPLAY error message

    RETURN result of calling menu() again

  RETURN choice

FUNCTION main()

  DISPLAY WeCare header

  SET products = load_products from file

  LOOP until exit

    SET choice = menu()

    IF choice is 1

```
        DISPLAY all products
    ELSE IF choice is 2
        DISPLAY add product header
        UPDATE products with add_product
    ELSE IF choice is 3
        DISPLAY sell product header
        UPDATE products with sell_product
    ELSE IF choice is 4
        DISPLAY exit message
        EXIT LOOP
    ELSE
        DISPLAY invalid choice message
  END LOOP

CALL main()
```

4 Flowchart

A graphical representation of workflow or process that shows steps, decision and sequences. Flowchart contain different shape for different function.

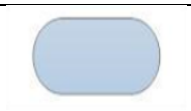
Shape	Name	Function
	Start/End	It represents start or end point.
	Arrows	To connect different shape to show relation.
	Input/Output	It represents input or output.
	Process	It represents the process involved.
	Decision	It is used to indicate the decision.

Table 1 Flowchart Basics

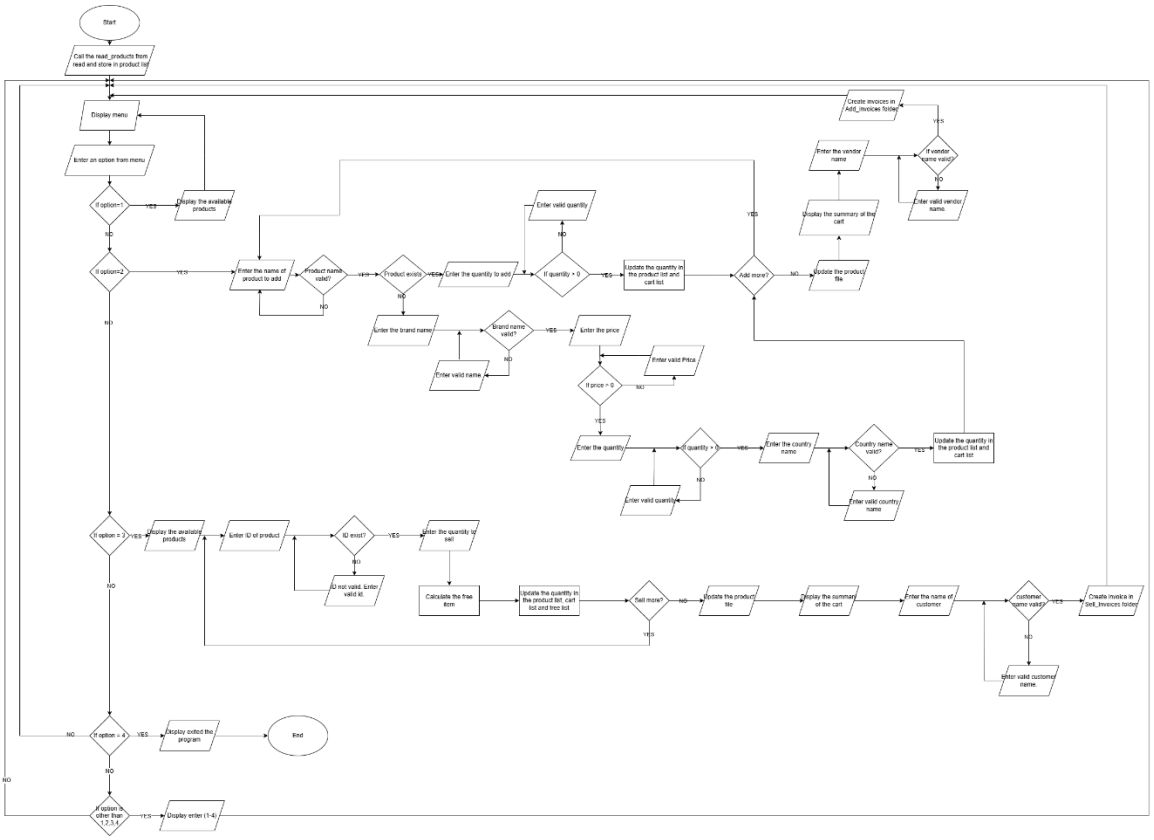


Figure 7 Flowchart

5 Data Structure

It is format for storing, managing and organizing data in a program. It is important to choose the right data structure while writing the code not just for the code to work but also to make the code easy to read, scale and maintain.

Python provides two different types of data structures:

- Primitive Data Structure
- Non-Primitive Data Structure

5.1 Primitive Data Structure

5.1.1 Integer

It is used to store whole numbers. Integer allows to perform arithmetic calculations (Jaiswal, 2023).

```
quantity = 100
cost_price = 800
```

Figure 8 Integer

5.1.2 Float

Float is used to store decimal numbers. It also allows to perform arithmetic calculations (Jaiswal, 2023).

```
total_price = 849.99
```

Figure 9 Float

5.1.3 String

String stores text-based data (Jaiswal, 2023).

```
brand = "Garnier"
origin = "France"
```

Figure 10 String

5.1.4 Boolean

Boolean store only two values i.e. true and false (Jaiswal, 2023).

```
in_stock = True
```

Figure 11 Boolean

5.2 Non-Primitive Data Structure

5.2.1 List

List are mutable (data can be changed) and can store multiple data of different data types (Jaiswal, 2023).

```
products = [
    ["Vitamin C Serum", "Garnier", 200, 1000, "France"],
    ["Skin Cleanser", "Cetaphil", 100, 280, "Switzerland"]
]
```

Figure 12 List

5.2.2 Tuple

Tuple are like list but are immutable (data cannot be changed) (Jaiswal, 2023).

```
promo_policy = (3, 1)
```

Figure 13 Tuple

5.2.3 Dictionary

It stores data in key-value pairs (Jaiswal, 2023).

```
product = {
    "name": "Vitamin C Serum",
    "brand": "Garnier",
    "price": 1000,
    "origin": "France"
}
```

Figure 14 Dictionary

5.2.4 Set

Set is an unordered collection of unique data (Jaiswal, 2023).

```
brands = {"Garnier", "Cetaphil", "Aqualogica"}
```

Figure 15 Set

5.3 Datatype Used

In the development of this project, I used different data types to store various kind of data effectively. The data type was chosen based on the nature of the information to be stored. The main data types that I used:

- String
- Integer
- List
- Boolean

5.3.1 String

The main purpose for using string was to store textual data such as product name, brand and vendor/customer name, because strings are best for storing readable name and can also be formatted easily.

```
name = str(input("Enter product name: "))
```

Figure 16 Datatype used (String)

Above is the example of use of string data type to store name of the product.

5.3.2 Integer

The use of integer was to store numeric value like quantity, price etc. because integer are very useful for arithmetic operations.

```
quantity = int(input("Enter product quantity to add: "))
```

Figure 17 Datatype used (Integer)

Above is an example of integer that I used to store quantity of the products.

5.3.3 List

The purpose of use of list was to store the product details and hold the group of details too, because of its flexibility to store multiple items together. And also, we can loop easily through each to modify them.

```
cart=[product[0], product[1], product[2], product[3], quantity, product[5]]
```

Figure 18 Datatype used (List)

Above is the example of using list to store the details in the cart for my project.

5.3.4 Boolean

The soul purpose for the use of Boolean was to store decision-making conditions like whether a product exists or not, because it is good for controlling logic with if-else conditions.

```
exist = False

if exist == False:
    id = str(len(products)+1)
    while True:
```

Figure 19 Datatype used (Boolean)

Above is the example of using Boolean to check if product exist or not.

6 Program

6.1 Code description

Here is the step-by-step explanation of my code what is does in an understandable form. It consists of four module each with different functions. Four modules:

1. Read.py
2. Write.py
3. Operation.py
4. Main.py

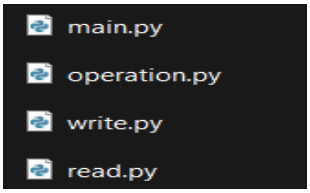
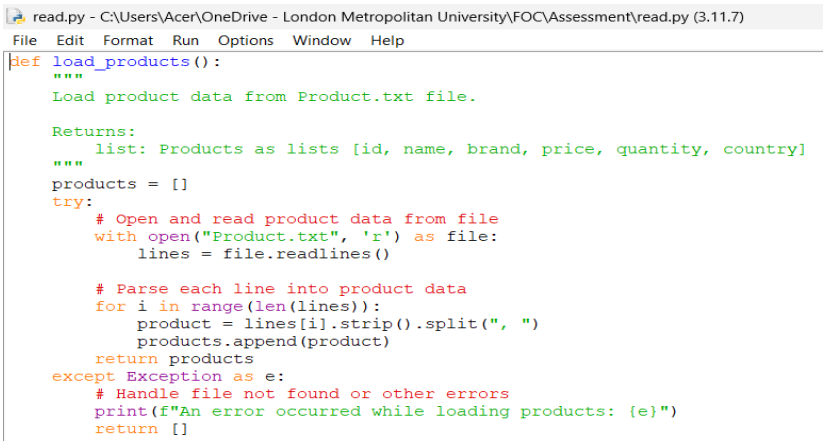


Figure 20 Modules

6.1.1 Read.py

The read module is used to load the product data from text file so that program can access the inventory. This module consists of only one function i.e. load_products(). This function opens the product file that contains the details of the product available in the inventory. It reads each line, split the individual data and store them as a list of lists. This function returns the list which is used throughout the program.




```
def load_products():
    """
    Load product data from Product.txt file.

    Returns:
        list: Products as lists [id, name, brand, price, quantity, country]
    """
    products = []
    try:
        # Open and read product data from file
        with open("Product.txt", 'r') as file:
            lines = file.readlines()

        # Parse each line into product data
        for i in range(len(lines)):
            product = lines[i].strip().split(", ")
            products.append(product)
        return products
    except Exception as e:
        # Handle file not found or other errors
        print(f"An error occurred while loading products: {e}")
        return []
```

Figure 21 function load_product

6.1.2 Write.py

The write module is used to save any chance made to the product in the inventory back to file and for creating invoices. It contains three functions:

1. Save_product
2. Add_invoices
3. Sell_invoices

A. Save_product function

It takes products list as argument and update the product file with the latest list of the products. It writes all the details of the product line by line to make sure that the current stocks is properly saved.

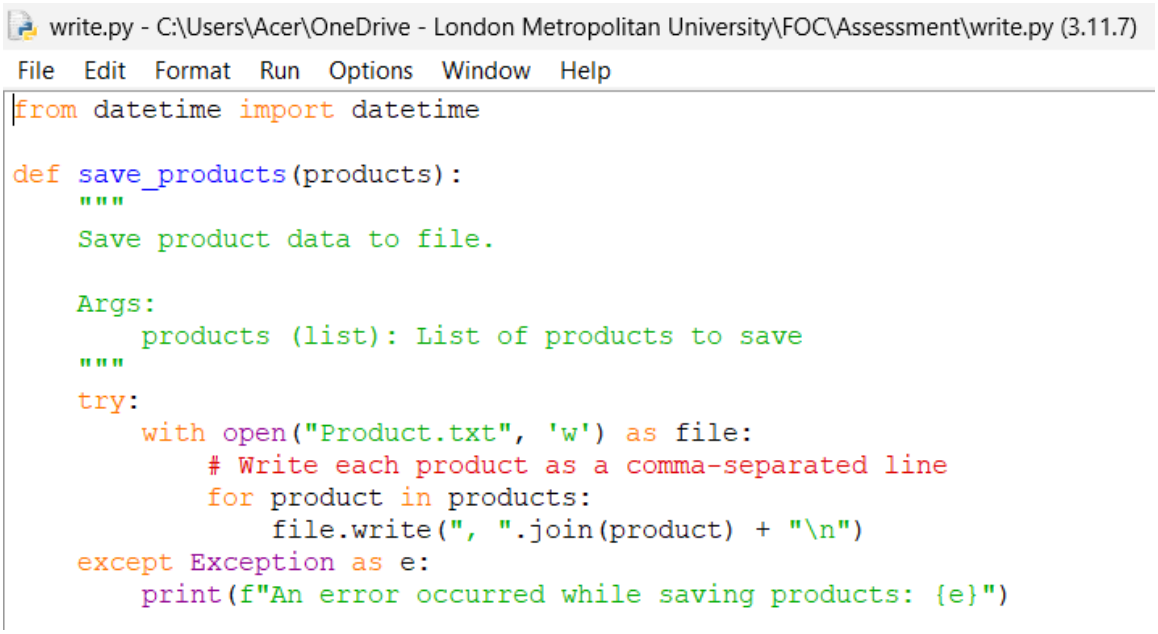


Figure 22 function save product

B. Add_invoices function

This function takes shopping_cart list and vendor's name as an argument and is used when products are added. It creates invoice as text file with unique name. This function is useful to track the purchase made for the store.

write.py - C:\Users\Acer\OneDrive - London Metropolitan University\FOC\Assessment\write.py (3.11.7)

File Edit Format Run Options Window Help

```
def add_invoices(shopping_cart, vendor):
    """
    Create invoice for added products.

    Args:
        shopping_cart (list): List of products added to inventory
        vendor (str): Name of the vendor supplying the products
    """

    # Generate unique invoice filename using timestamp
    invoice_name = str(datetime.now())
    invoice_name = invoice_name.replace(" ", "_")
    invoice_name = invoice_name.replace(":", "-")
    try:
        with open(f"Add_Invoices/{vendor}_{invoice_name}.txt", 'w') as file:
            # Create invoice header
            file.write("+" + "-" * 100 + "+\n")
            file.write(f"|{'INVOICE':^100}|\n")
            file.write(f"|{'WeCare Beauty Products':^100}|\n")
            file.write("+" + "-" * 100 + "+\n")
            file.write(f"| Name of Vendor: {vendor:<83}|\n")
            file.write(f"| Date of Purchase: {datetime.now().strftime('%Y-%m-%d %H:%M:%S'):<81}|\n")
            file.write("+" + "-" * 100 + "+\n")
            file.write(f"|{'ID':<5}| {'Name':<30}| {'Brand':<20}| {'Price':<10}| {'Quantity':<10}| {'Country':<15}|\n")
            file.write("+" + "-" * 100 + "+\n")

            # Calculate and write product details
            total_amount = 0
            for cart in shopping_cart:
                id = cart[0]
                name = cart[1][:29] if len(cart[1]) > 29 else cart[1]
                brand = cart[2][:19] if len(cart[2]) > 19 else cart[2]
                price = str(int(cart[3]))
                quantity = cart[4]
                country = cart[5][:14] if len(cart[5]) > 14 else cart[5]

                file.write(f"|{id:<5}| {name:<30}| {brand:<20}| {price:<10}| {quantity:<10}| {country:<15}|\n")

                total_amount += int(cart[3]) * int(quantity)

            # Write invoice footer with total amount
            file.write("+" + "-" * 100 + "+\n")
            vat = (13 / 100) * total_amount
            file.write(f"|{'Total Amount:':<58} {total_amount:<41}|\n")
            file.write(f"|{'Vat(13%):':<58} {vat:<41}|\n")
            file.write(f"|{'Grand Total:':<58} {(total_amount+vat):<41}|\n")
            file.write("+" + "-" * 100 + "+")
        print(f"Invoice created successfully")
    except Exception as e:
        print(f"An error occurred while creating the invoice: {e}")
```

Figure 23 function add invoices

C. Sell_invoices function

This is the last function of this module. It takes shopping_cart list, free list and customer name as an argument and is used when product are sold. It also creates invoices with unique name. This helps to record customer transactions for recordkeeping/references.

write.py - C:\Users\Acer\OneDrive - London Metropolitan University\FOC\Assessment\write.py (3.11.7)

File Edit Format Run Options Window Help

```
def sell_invoices(shopping_cart, customer, free):
    """
    Create invoice for sold products with free items.

    Args:
        shopping_cart (list): List of products sold
        customer (str): Name of the customer making the purchase
        free (list): List of free items given for each product
    """

    # Generate unique invoice filename using timestamp
    invoice_name = str(datetime.now())
    invoice_name = invoice_name.replace(" ", "_")
    invoice_name = invoice_name.replace(":", "-")
    try:
        with open(f"Sell_Invoices/{customer}_{invoice_name}.txt", 'w') as file:
            # Create invoice header
            file.write("+" + "-" * 117 + "+\n")
            file.write(f"|{'INVOICE':^117}|\n")
            file.write(f"|{'WeCare Beauty Products':^117}|\n")
            file.write("+" + "-" * 117 + "+\n")
            file.write(f"| Name of Customer: {customer:<98}|\n")
            file.write(f"| Date of Purchase: {datetime.now().strftime('%Y-%m-%d %H:%M:%S'):<98}|\n")
            file.write("+" + "-" * 117 + "+\n")
            file.write(f"|{'ID':<5}| {'Name':<25}| {'Brand':<15}| {'Price':<10}| {'Quantity':<10}| {'Free Item':<10}| {'Total Quantity':<15}| {'Country':<13}|\n")
            file.write("+" + "-" * 117 + "+\n")

            # Calculate and write product details with free items
            total_amount = 0
            for i, cart in enumerate(shopping_cart):
                id = cart[0]
                name = cart[1][:29] if len(cart[1]) > 29 else cart[1]
                brand = cart[2][:19] if len(cart[2]) > 19 else cart[2]
                price = str(int(cart[3]))
                quantity = cart[4]
                country = cart[5][:14] if len(cart[5]) > 14 else cart[5]
                free_item = free[i]

                file.write(f"|{id:<5}| {name:<25}| {brand:<15}| {price:<10}| {(int(quantity)-int(free_item)):<10}| {free_item:<10}| {int(quantity):<15}| {country:<13}|\n")

                total_amount += int(cart[3]) * int(int(quantity)-free_item)

            # Write invoice footer with total amount
            file.write("+" + "-" * 117 + "+\n")
            vat = (13 / 100) * total_amount
            file.write(f"|{'Total Amount:':<58} {total_amount:<58}|\n")
            file.write(f"|{'Vat(13%):':<58} {vat:<58}|\n")
            file.write(f"|{'Grand Total:':<58} {(total_amount+vat):<58}|\n")
            file.write("+" + "-" * 117 + "+")
        print(f"Invoice created successfully")
```

Figure 24 function sell invoices

6.1.3 Operation.py

This module is the most important part of the system. It contains the entire logic how products are displayed, added and sold. There are basically five functions in this module. Five functions:

1. Display_products
2. Add_summary
3. Sell_summary
4. Add_product
5. Sell_product

A. Display_products function

This function takes product list as an argument and prints all the products with their details such as id, name, price, quantity etc. on a formatted table. It also doubles the price of the product while displaying to simulate the selling price.

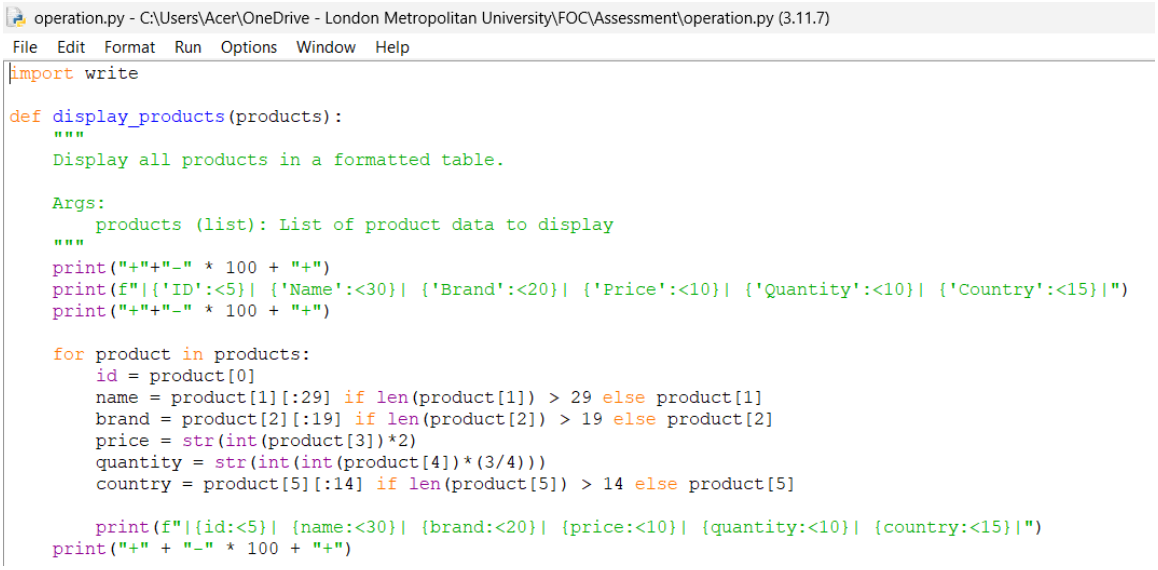


Figure 25 function display products

B. Add_summary function

This function takes shopping_cart list as an argument and is used after the product is added to the inventory. It shows the summary table of the product that were just added. It also calculates the total price for the reference.

```
def add_summary(shopping_cart):
    """
    Display a summary of products added to the cart.

    Args:
        shopping_cart (list): List of products in the cart
    """
    print("+" + "-" * 100 + "+")
    print(f"|{'Checkout Summary':^100}|")
    print("+" + "-" * 100 + "+")
    print(f"|{'Name of Product':<50}| {'Price':<22}| {'Quantity':<24}| ")
    print("+" + "-" * 100 + "+")
    total=0
    for cart in shopping_cart:
        name = cart[1][:49] if len(cart[1]) > 49 else cart[1]
        price = str(int(cart[3]))
        quantity = cart[4]
        print(f"|{name:<50}| {price:<22}| {quantity:<24}|")
        total += int(cart[3])*int(cart[4])
    print("+" + "-" * 100 + "+")
    print(f"|{'Total Amount:':<50} {total:<49}|")
    print("+" + "-" * 100 + "+")
```

Figure 26 function add summary

C. Sell_summary function

This function takes shopping_cart list and free list as arguments and is used after selling the product. It shows the summary table including id, name, quantity, free items and total quantity. It also calculate the amount after applying the “Buy 3 Get 1 Free” offer.

```
def sell_summary(shopping_cart, free):
    """
    Display a summary of products sold with free items.

    Args:
        shopping_cart (list): List of products sold
        free (list): List of free items per product
    """
    print("+" + "-" * 100 + "+")
    print(f"|{'Checkout Summary':^100}|")
    print("+" + "-" * 100 + "+")
    print(f"|{'Name of Product':<50}| {'Price':<7}| {'Quantity':<10}| {'Free Item':<10}| {'Total Quantity':<15}|")
    print("+" + "-" * 100 + "+")
    total=0

    for i,cart in enumerate(shopping_cart):
        free_item=free[i]
        name = cart[1][:49] if len(cart[1]) > 49 else cart[1]
        price = str(int(cart[3]))
        quantity = cart[4]
        print(f"|{name:<50}| {price:<7}| {(int(quantity)-free_item):<10}| {free_item:<10}| {int(quantity):<15}|")
        total += int(cart[3])*int(int(cart[4])-free_item)
    print("+" + "-" * 100 + "+")
    print(f"|{'Total Amount:':<59} {total:<40}|")
    print("+" + "-" * 100 + "+")
```

Figure 27 function sell summary

D. Add_product function

This function takes products list as an argument and allows user to either add quantity of the existing product or add details of the entirely new product to add in the inventory. It also includes the validation of the inputs to prevents errors. After adding the products, it shows summary and create invoices.

```
def add_product(products):
    """
    Add new products or update quantities of existing products.

    This function allows:
    - Adding quantity to existing products
    - Creating new products with details
    - Generating invoices for the transaction

    Args:
        products (list): Current list of products

    Returns:
        list: Updated list of products
    """
    shopping_cart=[]
    while True:
        exist = False
        # Get and validate product name
        while True:
            try:
                name = str(input("Enter product name: "))
                name = name.upper()
                if len(name) == 0:
                    raise ValueError("Product name cannot be empty.")
                break
            except ValueError as e:
                if "Product name cannot be empty." in str(e):
                    print(f"Invalid input: {e}")
                else:
                    print("Invalid input. Please enter a valid product name.")

        # Check if product exists and update quantity
        for product in products:
            if product[1].strip() == name.strip():
                while True:
                    try:
                        quantity = int(input("Enter product quantity to add: "))
                        if int(quantity) <= 0:
                            raise ValueError("Enter positive integer.")
                        elif int(quantity) > 0:
                            quantity = str(int(quantity))
                            break
                    except ValueError as e:
                        if "Enter positive integer." in str(e):
                            print(f"Invalid input: {e}")
                        else:
                            print("Invalid input. Please enter a valid integer.")

                product[4] = str(int(product[4]) + int(quantity))
```

Figure 28 function add product 1

```
                product[4] = str(int(product[4]) + int(quantity))
                exist = True
                cart=[product[0], product[1], product[2], product[3], quantity, product[5]]
                shopping_cart.append(cart)
                break

        # If product doesn't exist, get details and add new product
        if exist == False:
            id = str(len(products)+1)
            while True:
                try:
                    brand = str(input("Enter product brand: "))
                    brand = brand.upper()
                    if len(brand) == 0:
                        raise ValueError("Brand name cannot be empty.")
                    break
                except ValueError as e:
                    if "Brand name cannot be empty." in str(e):
                        print(f"Invalid input: {e}")
                    else:
                        print("Invalid input. Please enter a valid brand name.")

            while True:
                try:
                    price = int(input("Enter product price: "))
                    if int(price) < 0:
                        raise ValueError("Enter positive integer.")
                    elif int(price) > 0:
                        price = str(int(price))
                        break
                except ValueError as e:
                    if "Enter positive integer." in str(e):
                        print(f"Invalid input: {e}")
                    else:
                        print("Invalid input. Please enter a valid integer.")

            while True:
                try:
                    quantity = int(input("Enter product quantity: "))
                    if int(quantity) < 0:
                        raise("Enter positive integer.")
                    elif int(quantity) > 0:
                        quantity = str(int(quantity))
                        break
                except ValueError as e:
                    if "Enter positive integer." in str(e):
                        print(f"Invalid input: {e}")
                    else:
                        print("Invalid input. Please enter a valid integer.")
```

Figure 29 function add product 2

```
                    print("Invalid input. Please enter a valid integer.")

            while True:
                try:
                    country = str(input("Enter product country: "))
                    country = country.upper()
                    if len(country) == 0:
                        raise ValueError("Country name cannot be empty.")
                    for char in country:
                        if not(char.isalpha() or char.isspace()):
                            raise ValueError("Enter valid country name.")
                    break
                except ValueError as e:
                    if "Country name cannot be empty." in str(e):
                        print(f"Invalid input: {e}")
                    elif "Enter valid country name." in str(e):
                        print(f"Invalid input: {e}")
                    else:
                        print("Invalid input. Please enter a valid country name.")
            products.append([id, name, brand, price, quantity, country])
            cart=[id, name, brand, price, quantity, country]
            shopping_cart.append(cart)

        # Ask if user wants to add another product
        while True:
            continue_shopping = input("Do you want to add another product? (Y/N): ").strip().upper()
            if continue_shopping == 'N':
                break
            elif continue_shopping == 'Y':
                break
            else:
                print("Invalid choice. Please enter Y or N.")

        if continue_shopping == 'N':
            break

write.save_products(products)
add_summary(shopping_cart)

# Get and validate vendor name
while True:
    try:
        vendor = str(input("Enter the name of vendor: "))
        vendor = vendor.upper()
        if len(vendor) == 0:
            raise ValueError("Vendor name cannot be empty.")
        for char in vendor:
            if not(char.isalpha() or char.isspace()):
                raise ValueError("Enter valid vendor name.")
        break
```

Figure 30 function add product 3

```
                raise ValueError("Enter valid vendor name.")
            break
        except ValueError as e:
            if "Enter valid vendor name." in str(e):
                print(f"Invalid input: {e}")
            elif "Vendor name cannot be empty." in str(e):
                print(f"Invalid input: {e}")
            else:
                print("Invalid input. Please enter a valid vendor name.")

    write.add_invoices(shopping_cart,vendor)
    print("Product added successfully.")
    return products
```

Figure 31 function add product 4

E. Sell_product function

This function takes products list as an argument and allow user to sell product by entering product's id and quantity that customer want to buy. It checks the availability of the quantity, calculate the free items and reduce the quantity in the inventory. After selling the product, it shows summary and create invoices.

operation.py - C:\Users\Acer\OneDrive - London Metropolitan University\FOC\Assessment\operation.py (3.11.7)

File Edit Format Run Options Window Help

```
def sell_product(products):
    """
    Sell products with buy 3 get 1 free promotion.

    This function handles:
    - Product selection by ID
    - Quantity validation and stock checking
    - Free item calculation (buy 3 get 1 free)
    - Multiple product purchase in one transaction
    - Invoice generation

    Args:
        products (list): Current list of products

    Returns:
        list: Updated list of products after sales
    """
    shopping_cart=[]
    free=[]
    display_products(products)

    while True:
        product_id = input("Enter the product ID to buy: ").strip()
        exist = False
        quantitynotFound = False
        # Find product by ID and handle purchase
        for product in products:
            if product[0] == product_id:
                exist = True
                while True:
                    try:
                        quantity = int(input("Enter quantity to buy: ").strip())
                        free_item=int(quantity)//3
                        available_quantity=int(int(product[4])*(3/4))
                        if int(quantity) <= 0:
                            print("Invalid quantity. Please enter a positive number.")
                        elif available_quantity == 0:
                            quantity = 0
                            free_item = 0
                            print("Product is out of stock.")
                            quantitynotFound = True
                            break
                        elif available_quantity < quantity:
                            print(f"Insufficient stock. Available quantity is {available_quantity}.")
                        elif available_quantity >= quantity:
                            break
                    except ValueError:
                        print("Invalid input. Please enter a valid integer.")
                # Update inventory and add to cart
                product[4] = str(int(product[4]) - int(quantity+free_item))
```

Figure 32 function sell product 1

operation.py - C:\Users\Acer\OneDrive - London Metropolitan University\FOC\Assessment\operation.py (3.11.7)

File Edit Format Run Options Window Help

```
                # Update inventory and add to cart
                product[4] = str(int(product[4]) - int(quantity+free_item))
                cart=[product[0], product[1], product[2], int(product[3])*2, str(int(quantity)+free_item), product[5]]
                free.append(free_item)
                shopping_cart.append(cart)
                break

        if exist == False:
            print("Product not found. Select a valid product ID.")
            continue

        # Ask if user wants to buy another product
        if quantitynotFound == True or quantitynotFound == False:
            while True:
                continue_shopping = input("Do you want to buy another product? (Y/N): ").strip().upper()
                if continue_shopping == 'N':
                    break
                elif continue_shopping == 'Y':
                    break
                else:
                    print("Invalid choice. Please enter Y or N.")

            if continue_shopping == 'N':
                break

    i = 0
    while i < len(shopping_cart):
        if shopping_cart[i][4] == '0':
            shopping_cart.pop(i)
            free.pop(i)
            # Don't increment i since the next item has shifted to this position
        else:
            i += 1  # Move to next item only if we didn't remove anything

    if len(shopping_cart) == 0:
        print("no product bought.")
        return products

    elif len(shopping_cart) >= 1:
        # Save changes and generate invoice
        write.save_products(products)
        sell_summary(shopping_cart,free)
```

Figure 33 function sell product 2

```
        write.save_products(products)
        sell_summary(shopping_cart,free)

        # Get and validate customer name - must contain valid characters only
        while True:
            try:
                customer = str(input("Enter the name of customer: "))
                customer = customer.upper()
                if len(customer) == 0:
                    raise ValueError("Customer name cannot be empty.")
                for char in customer:
                    if not(char.isalpha() or char.isspace()):
                        raise ValueError("Enter valid customer name.")
                break
            except ValueError as e:
                if "Enter valid customer name." in str(e):
                    print(f"Invalid input: {e}")
                else:
                    print("Invalid input. Please enter a valid customer name.")

        # Generate invoice and confirm purchase
        write.sell_invoices(shopping_cart,customer,free)
        print("Product bought successfully.")
        return products
```

Figure 34 function sell product 3

6.1.4 Main.py

This module is the entry point of the program. It controls how the user interact with the system by showing a menu with options. It has two function:

1. Menu
2. Main

A. Menu function

It shows the user different options to display products, add product, sell product oe exit. It collects the user input and return it to main function.

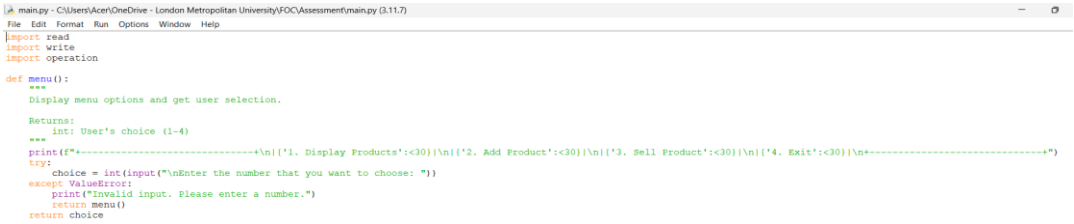


Figure 35 function menu

B. Main function

This functions start the program by loading the products using read module. It then show menu continuously according to the user's choice and call the right function from operation module. If the user's choice is to exit the progeam it ends the loop and close the program.

```
def main():
    """
    Main program function that runs the WeCare inventory system.

    Loads product data, displays menu options, and processes user choices
    until the user chooses to exit.

    Returns:
        None
    """
    print(f"+------------------------------+\n|{'WeCare':<30}|")
    products = read.load_products()

    while True:
        choice = menu()

        if choice == 1:
            operation.display_products(products)
        elif choice == 2:
            print(f"\n+-------------------------+\n|{'Add New Product':<25}|\n+-------------------------+\n")
            products = operation.add_product(products)
        elif choice == 3:
            print(f"\n+-------------------------+\n|{'Sell Product':<25}|\n+-------------------------+")
            products = operation.sell_product(products)
        elif choice == 4:
            print("\nExited the Program!\n")
            break
        else:
            print("\nInvalid choice. Please try again.\n")

main()
```

Figure 36 function main

6.2 Program flow

The program start by displaying the title WeCare and loads the existing product data from text file. After this, program enter the loop where a menu appear consisting of 4 choices: Display Products, Add Product, Sell Product and Exit. The user enters the number associated with the choices.

If the user select to display products, the program prints the details of the available products in a tabular format. If the user want to add product, the program either update the quantity of the product or add new one according to the name provided and then update the product list and generate summary and vendor invoice. If the user select to sell product, the program handle sell applying Buy 3 Get 1 Free offer, calculate the total, update the product list and creates summary and customer invoice. After each of the following actions, the menu is shown again, allowing user to perform multiple task at a time. The program continues until the user choose to exit the program.

6.2.1 Display

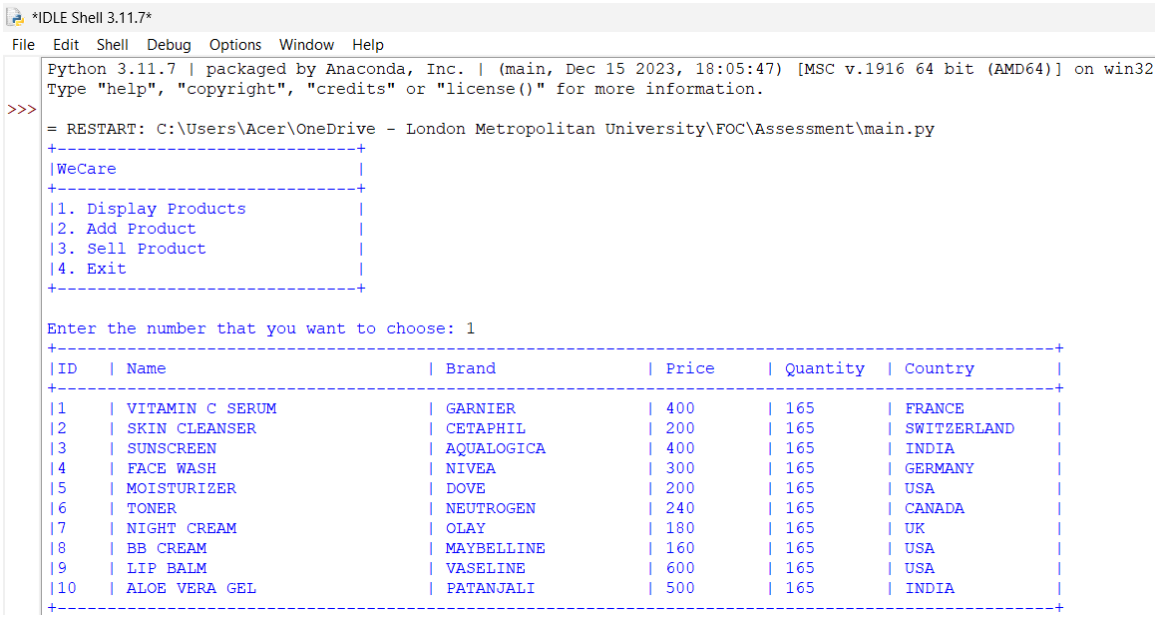


Figure 37 Display output

6.2.2 Add product

```
Enter the number that you want to choose: 2

+-------------------------+
|Add New Product          |
+-------------------------+

Enter product name: aloe vera gel
Enter product quantity to add: 20
Do you want to add another product? (Y/N): n
+-------------------------------------------------------------------------------------------------------+
|                                        Checkout Summary                                               |
+-------------------------------------------------------------------------------------------------------+
|Name of Product                                  | Price                | Quantity                |
+-------------------------------------------------------------------------------------------------------+
|ALOE VERA GEL                                    | 250                  | 20                      |
+-------------------------------------------------------------------------------------------------------+
|Total Amount:                                     5000                                                 |
+-------------------------------------------------------------------------------------------------------+
Enter the name of vendor: IIC
Invoice created successfully
Product added successfully.
```

Figure 38 Add product output

6.2.3 Sell product

```
Enter the number that you want to choose: 3

+-------------------------+
|Sell Product             |
+-------------------------+
+-------------------------------------------------------------------------------------------------------+
|ID   | Name                          | Brand              | Price     | Quantity  | Country         |
+-------------------------------------------------------------------------------------------------------+
|1    | VITAMIN C SERUM               | GARNIER            | 400       | 165       | FRANCE          |
|2    | SKIN CLEANSER                 | CETAPHIL           | 200       | 165       | SWITZERLAND     |
|3    | SUNSCREEN                     | AQUALOGICA         | 400       | 165       | INDIA           |
|4    | FACE WASH                     | NIVEA              | 300       | 165       | GERMANY         |
|5    | MOISTURIZER                   | DOVE               | 200       | 165       | USA             |
|6    | TONER                         | NEUTROGEN          | 240       | 165       | CANADA          |
|7    | NIGHT CREAM                   | OLAY               | 180       | 165       | UK              |
|8    | BB CREAM                      | MAYBELLINE         | 160       | 165       | USA             |
|9    | LIP BALM                      | VASELINE           | 600       | 165       | USA             |
|10   | ALOE VERA GEL                 | PATANJALI          | 500       | 180       | INDIA           |
+-------------------------------------------------------------------------------------------------------+
Enter the product ID to buy: 10
Enter quantity to buy: 15
Do you want to buy another product? (Y/N): n
+-------------------------------------------------------------------------------------------------------+
|                                        Checkout Summary                                               |
+-------------------------------------------------------------------------------------------------------+
|Name of Product                                | Price  | Quantity  | Free Item | Total Quantity |
+-------------------------------------------------------------------------------------------------------+
|ALOE VERA GEL                                  | 500    | 15        | 5         | 20             |
+-------------------------------------------------------------------------------------------------------+
|Total Amount:                                              7500                                        |
+-------------------------------------------------------------------------------------------------------+
Enter the name of customer: Sugam
Invoice created successfully
Product sold successfully.
```

Figure 39 Sell product output

6.2.4 Exit

```
Enter the number that you want to choose: 4

Exited the Program!

>>>
```

Figure 40 Exit output

6.3 Error Handling

While creating this program the error handling played a crucial role in ensuring that the program works without any interruption when the user enter the wrong input. I used try-except block to manage the error, which helped to protect the program from unexpected crashes like if the user enter the non-integer value or negative integer in the price or quantity section the program ask the user to enter the positive integer value. This made the program more user friendly and ensure the smooth operation without interruption. One of the example is given below:

```
try:
    price = int(input("Enter product price: "))
    if int(price) < 0:
        raise ValueError("Enter positive integer.")
    elif int(price) > 0:
        price = str(int(price))
        break
except ValueError as e:
    if "Enter positive integer." in str(e):
        print(f"Invalid input: {e}")
    else:
        print("Invalid input. Please enter a valid integer.")
```

Figure 41 Error Handling Example (Code)

```
+-------------------------+
|Add New Product          |
+-------------------------+

Enter product name: New Cream
Enter product brand: zero
Enter product price: -23
Invalid input: Enter positive integer.
Enter product price:
```

Figure 42 Error Handling Example (Output)

In the above example when user entered negative integer in price the program asked user to enter positive integer, which prevented the program from logical error.

7 Testing

7.1 Test 1

To test the display option.

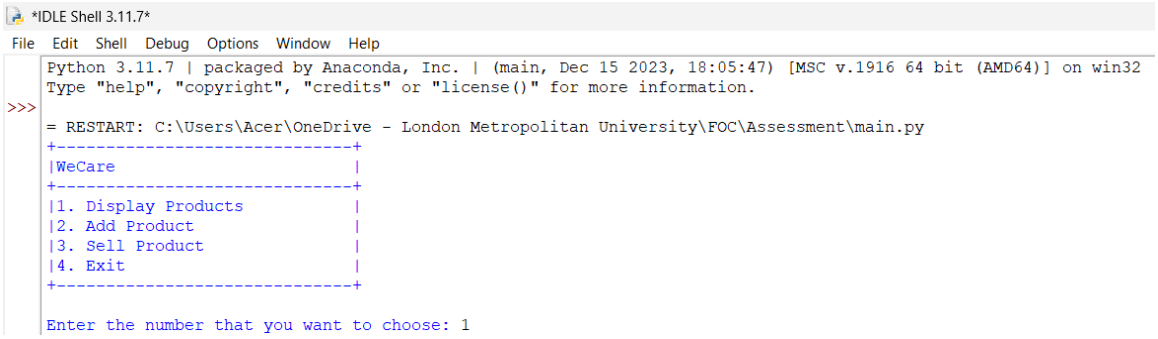


Figure 43 Display option test

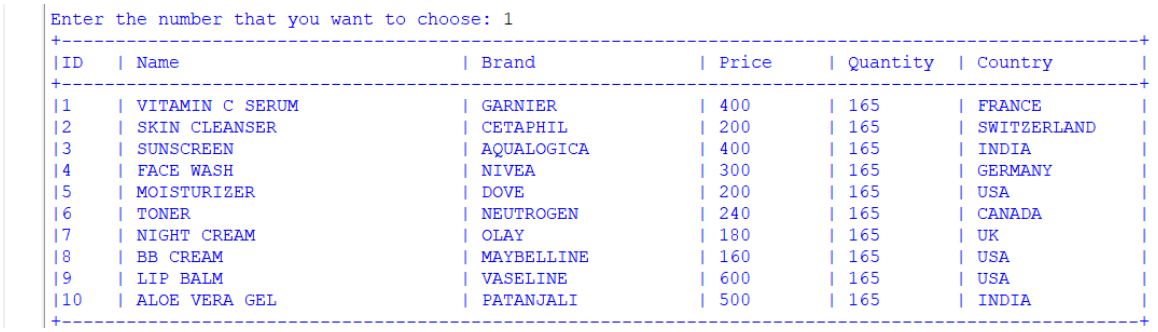


Figure 44 Display option test (output)

Objective	To test the display option
Action	When program asked to choose the number, 1 is entered
Expected Output	All the product with details should be shown.
Actual Output	All the product with their details were shown properly.
Result	The test was successful.

Table 2 Display option test

7.2 Test 2

To check the add product option.

7.2.1 Existing product

```
*IDLE Shell 3.11.7*
File  Edit  Shell  Debug  Options  Window  Help
Python 3.11.7 | packaged by Anaconda, Inc. | (main, Dec 15 2023, 18:05:47) [MSC v.1916 64 bit (AMD64)] on win32
Type "help", "copyright", "credits" or "license()" for more information.
>>>
= RESTART: C:\Users\Acer\OneDrive - London Metropolitan University\FOC\Assessment\main.py
+------------------------------+
|WeCare                        |
+------------------------------+
|1. Display Products           |
|2. Add Product                |
|3. Sell Product               |
|4. Exit                       |
+------------------------------+

Enter the number that you want to choose: 1
+-----------------------------------------------------------------------------------------------------+
|ID   | Name                        | Brand              | Price     | Quantity  | Country         |
+-----------------------------------------------------------------------------------------------------+
|1    | VITAMIN C SERUM             | GARNIER            | 400       | 165       | FRANCE          |
|2    | SKIN CLEANSER               | CETAPHIL           | 200       | 165       | SWITZERLAND     |
|3    | SUNSCREEN                   | AQUALOGICA         | 400       | 165       | INDIA           |
|4    | FACE WASH                   | NIVEA              | 300       | 165       | GERMANY         |
|5    | MOISTURIZER                 | DOVE               | 200       | 165       | USA             |
|6    | TONER                       | NEUTROGEN          | 240       | 165       | CANADA          |
|7    | NIGHT CREAM                 | OLAY               | 180       | 165       | UK              |
|8    | BB CREAM                    | MAYBELLINE         | 160       | 165       | USA             |
|9    | LIP BALM                    | VASELINE           | 600       | 165       | USA             |
|10   | ALOE VERA GEL               | PATANJALI          | 500       | 165       | INDIA           |
+-----------------------------------------------------------------------------------------------------+
+------------------------------+
|1. Display Products           |
|2. Add Product                |
|3. Sell Product               |
|4. Exit                       |
+------------------------------+

Enter the number that you want to choose: 2

+-------------------------+
|Add New Product          |
+-------------------------+

Enter product name: aloe vera gel
Enter product quantity to add: 200
Do you want to add another product? (Y/N): n
```

Figure 45 Adding existing product

```
Enter the number that you want to choose: 1
+-----------------------------------------------------------------------------------------------------+
|ID   | Name                        | Brand              | Price     | Quantity  | Country         |
+-----------------------------------------------------------------------------------------------------+
|1    | VITAMIN C SERUM             | GARNIER            | 400       | 165       | FRANCE          |
|2    | SKIN CLEANSER               | CETAPHIL           | 200       | 165       | SWITZERLAND     |
|3    | SUNSCREEN                   | AQUALOGICA         | 400       | 165       | INDIA           |
|4    | FACE WASH                   | NIVEA              | 300       | 165       | GERMANY         |
|5    | MOISTURIZER                 | DOVE               | 200       | 165       | USA             |
|6    | TONER                       | NEUTROGEN          | 240       | 165       | CANADA          |
|7    | NIGHT CREAM                 | OLAY               | 180       | 165       | UK              |
|8    | BB CREAM                    | MAYBELLINE         | 160       | 165       | USA             |
|9    | LIP BALM                    | VASELINE           | 600       | 165       | USA             |
|10   | ALOE VERA GEL               | PATANJALI          | 500       | 315       | INDIA           |
+-----------------------------------------------------------------------------------------------------+
```

Figure 46 Adding existing product (output)

Objective	To add the existing product
Action	The quantity to be increased by is entered
Expected Output	The quantity should be increased by the multiple of entered quantity and 0.75.
Actual Output	The quantity is increased by the multiple of entered quantity and 0.75.
Result	The test was successful.

Table 3 Adding existing product

7.2.2 New product

```
Enter the number that you want to choose: 2

+-------------------------+
|Add New Product          |
+-------------------------+

Enter product name: lip stick
Enter product brand: maybelline
Enter product price: 100
Enter product quantity: 200
Enter product country: USA
```

Figure 47 Adding new product

```
Enter the number that you want to choose: 1
+-----------------------------------------------------------------------------------------------------+
|ID   | Name                        | Brand              | Price    | Quantity  | Country          |
+-----------------------------------------------------------------------------------------------------+
|1    | VITAMIN C SERUM             | GARNIER            | 400      | 165       | FRANCE           |
|2    | SKIN CLEANSER               | CETAPHIL           | 200      | 165       | SWITZERLAND      |
|3    | SUNSCREEN                   | AQUALOGICA         | 400      | 165       | INDIA            |
|4    | FACE WASH                   | NIVEA              | 300      | 165       | GERMANY          |
|5    | MOISTURIZER                 | DOVE               | 200      | 165       | USA              |
|6    | TONER                       | NEUTROGEN          | 240      | 165       | CANADA           |
|7    | NIGHT CREAM                 | OLAY               | 180      | 165       | UK               |
|8    | BB CREAM                    | MAYBELLINE         | 160      | 165       | USA              |
|9    | LIP BALM                    | VASELINE           | 600      | 165       | USA              |
|10   | ALOE VERA GEL               | PATANJALI          | 500      | 315       | INDIA            |
|11   | LIP STICK                   | MAYBELLINE         | 200      | 150       | USA              |
+-----------------------------------------------------------------------------------------------------+
```

Figure 48 Adding new product (output)

Objective	To add new product
Action	All the details of the new product was added.
Expected Output	The product should be added to the product list.
Actual Output	The product is added to the product list.
Result	The test was successful.

Table 4 Adding new product

7.3 Test 3

To check the validity of quantity in adding product

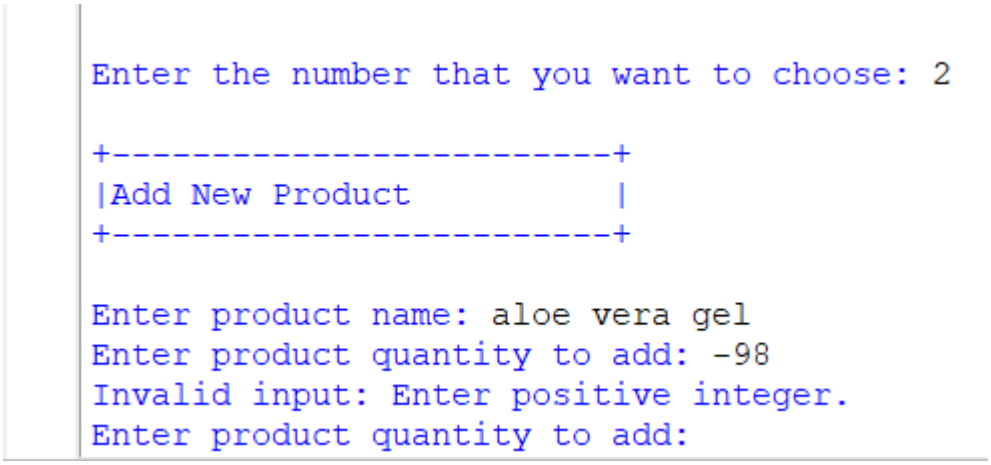


Figure 49 Quantity validity in add product

Objective	To check the validity of quantity while adding product.
Action	The quantity was filled negative.
Expected Output	The program should ask the user to enter positive integer.
Actual Output	The program asked to enter positive integer.
Result	The test was successful.

Table 5 Quantity validity in add product

7.4 Test 4

To check the sell product option

```
Enter the number that you want to choose: 3

+-------------------------+
|Sell Product             |
+-------------------------+
+-----------------------------------------------------------------------------------------------------+
|ID   | Name                        | Brand               | Price     | Quantity  | Country         |
+-----------------------------------------------------------------------------------------------------+
|1    | VITAMIN C SERUM             | GARNIER             | 400       | 145       | FRANCE          |
|2    | SKIN CLEANSER               | CETAPHIL            | 200       | 165       | SWITZERLAND     |
|3    | SUNSCREEN                   | AQUALOGICA          | 400       | 165       | INDIA           |
|4    | FACE WASH                   | NIVEA               | 300       | 165       | GERMANY         |
|5    | MOISTURIZER                 | DOVE                | 200       | 165       | USA             |
|6    | TONER                       | NEUTROGEN           | 240       | 165       | CANADA          |
|7    | NIGHT CREAM                 | OLAY                | 180       | 165       | UK              |
|8    | BB CREAM                    | MAYBELLINE          | 160       | 165       | USA             |
|9    | LIP BALM                    | VASELINE            | 600       | 165       | USA             |
|10   | ALOE VERA GEL               | PATANJALI           | 500       | 315       | INDIA           |
|11   | LIP STICK                   | MAYBELLINE          | 200       | 150       | USA             |
+-----------------------------------------------------------------------------------------------------+
Enter the product ID to buy: 1
Enter quantity to buy: 20
Do you want to buy another product? (Y/N): n
+-----------------------------------------------------------------------------------------------------+
|                                        Checkout Summary                                             |
+-----------------------------------------------------------------------------------------------------+
|Name of Product                                | Price  | Quantity  | Free Item | Total Quantity |
+-----------------------------------------------------------------------------------------------------+
|VITAMIN C SERUM                                | 400    | 20        | 6         | 26             |
+-----------------------------------------------------------------------------------------------------+
|Total Amount:                                   8000                                                 |
+-----------------------------------------------------------------------------------------------------+
Enter the name of customer: Sugam
Invoice created successfully
Product sold successfully.
```

Figure 50 Selling product

Objective	To sell product
Action	The required field to sell product were filled.
Expected Output	The product sold successfully message should be shown.
Actual Output	The product sold successfully message is shown.
Result	The test was successful.

Table 6 Selling product

7.5 Test 5

To check quantity out of stock

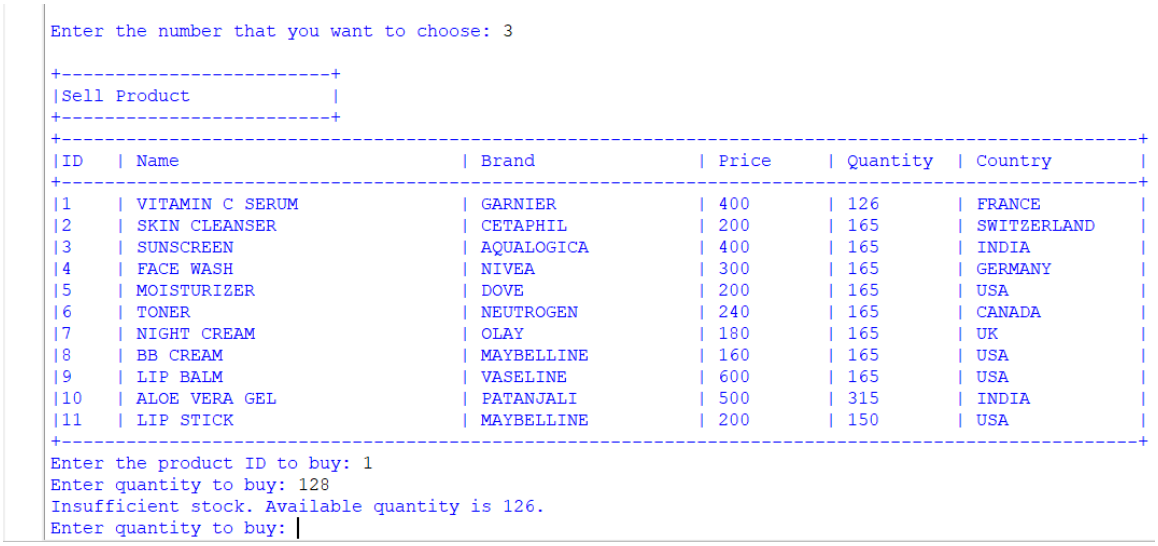


Figure 51 Quantity out of stock

Objective	To quantity out of stock
Action	The quantity is entered more than available.
Expected Output	The insufficient stock message should be shown.
Actual Output	The insufficient stock message is shown.
Result	The test was successful.

Table 7 Quantity out of stock

7.6 Test 6

To check buy 3 get 1 free offer

```
Enter the product ID to buy: 1
Enter quantity to buy: 20
Do you want to buy another product? (Y/N): n
+-------------------------------------------------------------------------------------------------------+
|                                        Checkout Summary                                               |
+-------------------------------------------------------------------------------------------------------+
|Name of Product                                | Price  | Quantity  | Free Item | Total Quantity |
+-------------------------------------------------------------------------------------------------------+
|VITAMIN C SERUM                                | 400    | 20        | 6         | 26             |
+-------------------------------------------------------------------------------------------------------+
|Total Amount:                                                 8000                                     |
+-------------------------------------------------------------------------------------------------------+
```

Figure 52 Buy 3 get 1 free

Objective	To check buy 3 get 1 free offer.
Action	Required field to sell product was filled.
Expected Output	The checkout summary with product details and free item should be shown.
Actual Output	The checkout summary with product details and free item is shown.
Result	The test was successful.

Table 8 Buy 3 get 1 free

7.7 Test 7

To check the add invoices is created.

```
Enter the number that you want to choose: 2

+-------------------------+
|Add New Product          |
+-------------------------+

Enter product name: aloe vera gel
Enter product quantity to add: 200
Do you want to add another product? (Y/N): n
+-------------------------------------------------------------------------------------------------------------+
|                                          Checkout Summary                                                   |
+-------------------------------------------------------------------------------------------------------------+
|Name of Product                                | Price                | Quantity                |
+-------------------------------------------------------------------------------------------------------------+
|ALOE VERA GEL                                  | 250                  | 200                     |
+-------------------------------------------------------------------------------------------------------------+
|Total Amount:                                   50000                                                        |
+-------------------------------------------------------------------------------------------------------------+
Enter the name of vendor: IIC
Invoice created successfully
Product added successfully.
```

Figure 53 Add invoice

```
IIC_2025-5-10-17-20-24-66541.txt
File    Edit    View

+-------------------------------------------------------------------------------------------------------------+
|                                          INVOICE                                                            |
|                                     WeCare Beauty Products                                                  |
+-------------------------------------------------------------------------------------------------------------+
| Name of Vendor: IIC                                                                                         |
| Date of Purchase: 2025-5-10 17:20:24                                                                        |
+-------------------------------------------------------------------------------------------------------------+
|ID   | Name                        | Brand              | Price     | Quantity  | Country        |
+-------------------------------------------------------------------------------------------------------------+
|10   | ALOE VERA GEL               | PATANJALI          | 250       | 200       | INDIA          |
+-------------------------------------------------------------------------------------------------------------+
|Total Amount:                                         50000                                                  |
|Vat(13%):                                             6500.0                                                 |
|Grand Total:                                          56500.0                                                |
+-------------------------------------------------------------------------------------------------------------+
```

Figure 54 Add invoice (output)

Objective	To check whether add invoice is created or not.
Action	The required field to add product was filled.
Expected Output	The invoice is created successfully message should be shown and invoice should be created in add_invoices folder.
Actual Output	The invoice is created successfully message was shown and invoices was created in add_invoices folder.
Result	The test was successful.

Table 9 Add invoice

7.8 Test 8

To check sell invoices is created.

```
Enter the product ID to buy: 1
Enter quantity to buy: 20
Do you want to buy another product? (Y/N): n
+-----------------------------------------------------------------------------------------------------------+
|                                          Checkout Summary                                                 |
+-----------------------------------------------------------------------------------------------------------+
|Name of Product                                | Price  | Quantity  | Free Item | Total Quantity |
+-----------------------------------------------------------------------------------------------------------+
|VITAMIN C SERUM                                | 400    | 20        | 6         | 26             |
+-----------------------------------------------------------------------------------------------------------+
|Total Amount:                                                  8000                                        |
+-----------------------------------------------------------------------------------------------------------+
Enter the name of customer: Sugam
Invoice created successfully
Product sold successfully.
```

Figure 55 Sell invoice

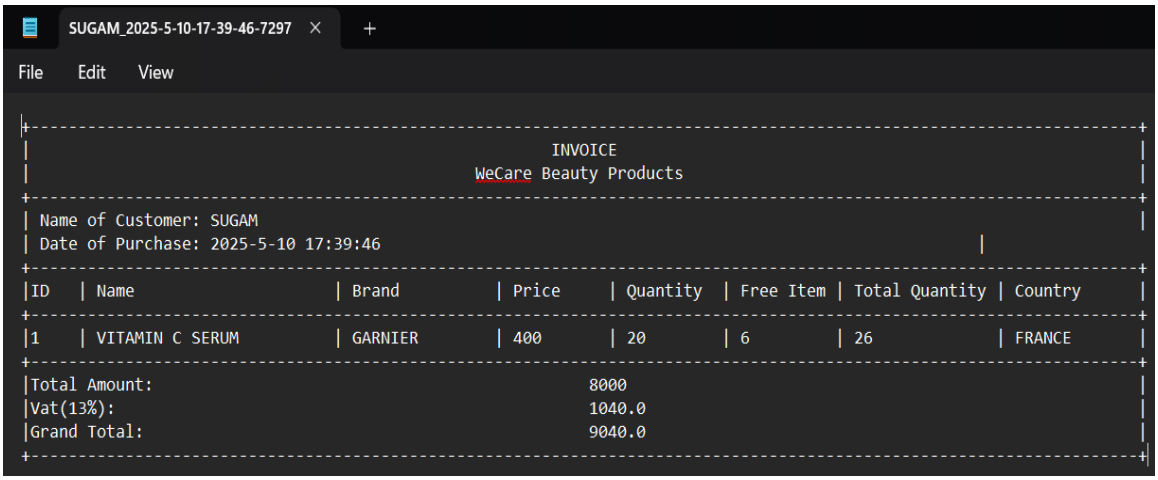


Figure 56 Sell invoice (output)

Objective	To check whether sell invoice is created or not.
Action	The required field to sell product was filled.
Expected Output	The invoice is created successfully message should be shown and invoice should be created in sell_invoices folder.
Actual Output	The invoice is created successfully message was shown and invoices was created in sell_invoices folder.
Result	The test was successful.

Table 10 Sell invoices

8 Conclusion

8.1 Fundamentals of Computing

Fundamentals of computing module introduced me to very essential concepts of computing algorithm, flowchart and basic python programming. As one who is very new to programming, this module helped me to get insight into how the programming world works and played a crucial on building my confidence towards this field too.

Throughout this module I learn to write and structure code, build logic and break problems into small pieces to solve them easily. This modules also helped me to build computational thinking and to know the importance of planning and testing during the development of any project.

8.2 Project (Coursework)

The project helped me to apply the concept of programming that I learned in class to build a real-life solution. While building the project I gained experience in file handling and modular programming in python.

I got the knowledge to read and organize the product data from text file, displaying in user-friendly format and adding the functionality to generate unique invoices and update the stocks automatically.

Overall, this project helped me to improve my problem-solving skill and gave me the foundation to build more advanced system.

9 References

Aezion, 2019. *Python Programming Language Introduction.* [Online] Available at: https://www.aezion.com/blogs/python-programming-language-introduction/
[Accessed 28 April 2025].

Grishina, A., 2023. *What is Pseudocode: A Complete Guide to Its Benefits and Use Cases.* [Online] Available at: https://softteco.com/blog/what-is-pseudocode
[Accessed 30 April 2025].

Jaiswal, S., 2023. *Python Data Structures Tutorial.* [Online] Available at: https://www.datacamp.com/tutorial/data-structures-python
[Accessed 28 April 2025].

Python Software Foundation, n.d.. *IDLE — Integrated Development and Learning Environment.* [Online] Available at: https://docs.python.org/3/library/idle.html
[Accessed 28 April 2025].

10 Appendix

10.1 Code

10.1.1 Read.py

```
def load_products():
    """
    Load product data from Product.txt file.


    Returns:
        list: Products as lists [id, name, brand, price, quantity, country]
    """
    products = []
    try:
        # Open and read product data from file
        with open("Product.txt", 'r') as file:
            lines = file.readlines()


        # Parse each line into product data
        for i in range(len(lines)):
            product = lines[i].strip().split(", ")
            products.append(product)
        return products
    except Exception as e:
        # Handle file not found or other errors
```

```
        print(f"An error occurred while loading products: {e}")
        return []
```

10.1.2 Write.py

```
from datetime import datetime


def save_products(products):
    """
    Save product data to file.

    Args:
        products (list): List of products to save
    """
    try:
        with open("Product.txt", 'w') as file:
            # Write each product as a comma-separated line
            for product in products:
                file.write(", ".join(product) + "\n")
    except Exception as e:
        print(f"An error occurred while saving products: {e}")


def add_invoices(shopping_cart, vendor):
    """
    Create invoice for added products.
```

```
    Args:
        shopping_cart (list): List of products added to inventory
        vendor (str): Name of the vendor supplying the products
    """

    # Generate unique invoice filename using timestamp
    invoice = datetime.now()
    invoice_name = str(invoice.year)+ "-" + str(invoice.month)+ "-" + str(invoice.day)+ "-" + str(invoice.hour)+ "-" + str(invoice.minute)+"-" + str(invoice.second)+"-"+ str(invoice.microsecond)
    dateOfPurchase = str(invoice.year)+"-"+str(invoice.month)+"-"+str(invoice.day)+" "+str(invoice.hour)+":"+str(invoice.minute)+":"+str(invoice.second)
    try:
        with open(f"Add_Invoices/{vendor}_{invoice_name}.txt", 'w') as file:
            # Create invoice header
            file.write("+" + "-" * 100 + "+\n")
            file.write(f"|{'INVOICE':^100}|\n")
            file.write(f"|{'WeCare Beauty Products':^100}|\n")
            file.write("+" + "-" * 100 + "+\n")
            file.write(f"| Name of Vendor: {vendor:<83}|\n")
            file.write(f"| Date of Purchase: {dateOfPurchase:<81}|\n")
            file.write("+" + "-" * 100 + "+\n")
```

```
        file.write(f"|{'ID':<5}| {'Name':<30}| {'Brand':<20}| {'Price':<10}| {'Quantity':<10}| {'Country':<15}|\n")
        file.write("+" + "-" * 100 + "+\n")

        # Calculate and write product details
        total_amount = 0
        for cart in shopping_cart:
            id = cart[0]
            name = cart[1][:29] if len(cart[1]) > 29 else cart[1]
            brand = cart[2][:19] if len(cart[2]) > 19 else cart[2]
            price = str(int(cart[3]))
            quantity = cart[4]
            country = cart[5][:14] if len(cart[5]) > 14 else cart[5]

            file.write(f"|{id:<5}| {name:<30}| {brand:<20}| {price:<10}| {quantity:<10}| {country:<15}|\n")

            total_amount += int(cart[3]) * int(quantity)

        # Write invoice footer with total amount
        file.write("+" + "-" * 100 + "+\n")
        vat = (13 / 100) * total_amount
        file.write(f"|{'Total Amount:':<58} {total_amount:<41}|\n")
```

```
            file.write(f"|{'Vat(13%):':<58} {vat:<41}|\n")
            file.write(f"|{'Grand Total:':<58} {(total_amount+vat):<41}|\n")
            file.write("+" + "-" * 100 + "+")
        print(f"Invoice created successfully")
    except Exception as e:
        print(f"An error occurred while creating the invoice: {e}")



def sell_invoices(shopping_cart, customer, free):
    """
    Create invoice for sold products with free items.

    Args:
        shopping_cart (list): List of products sold
        customer (str): Name of the customer making the purchase
        free (list): List of free items given for each product
    """

    # Generate unique invoice filename using timestamp
    invoice = datetime.now()
    invoice_name = str(invoice.year)+ "-" + str(invoice.month)+ "-" + str(invoice.day)+ "-" + str(invoice.hour)+ "-" + str(invoice.minute)+"-" + str(invoice.second)+"-"+ str(invoice.microsecond)
```

```
    dateOfPurchase = str(invoice.year)+"-"+str(invoice.month)+"-
"+str(invoice.day)+"
"+str(invoice.hour)+":"+str(invoice.minute)+":"+str(invoice.second)
    try:
        with open(f"Sell_Invoices/{customer}_{invoice_name}.txt", 'w') as file:
            # Create invoice header
            file.write("+" + "-" * 117 + "+\n")
            file.write(f"|{'INVOICE':^117}|\n")
            file.write(f"|{'WeCare Beauty Products':^117}|\n")
            file.write("+" + "-" * 117 + "+\n")
            file.write(f"| Name of Customer: {customer:<98}|\n")
            file.write(f"| Date of Purchase: {dateOfPurchase:<98}|\n")
            file.write("+" + "-" * 117 + "+\n")
            file.write(f"|{'ID':<5}| {'Name':<25}| {'Brand':<15}| {'Price':<10}|
{'Quantity':<10}| {'Free Item':<10}| {'Total Quantity':<15}| {'Country':<13}|\n")
            file.write("+" + "-" * 117 + "+\n")

            # Calculate and write product details with free items
            total_amount = 0
            for i, cart in enumerate(shopping_cart):
                id = cart[0]
                name = cart[1][:29] if len(cart[1]) > 29 else cart[1]
                brand = cart[2][:19] if len(cart[2]) > 19 else cart[2]
```

```
            price = str(int(cart[3]))
            quantity = cart[4]
            country = cart[5][:14] if len(cart[5]) > 14 else cart[5]
            free_item = free[i]

            file.write(f"|{id:<5}| {name:<25}| {brand:<15}| {price:<10}| {(int(quantity)-int(free_item)):<10}| {free_item:<10}| {int(quantity):<15}| {country:<13}|\n")

            total_amount += int(cart[3]) * int(int(quantity)-free_item)

        # Write invoice footer with total amount
        file.write("+" + "-" * 117 + "+\n")
        vat = (13 / 100) * total_amount
        file.write(f"|{'Total Amount:':<58} {total_amount:<58}|\n")
        file.write(f"|{'Vat(13%):':<58} {vat:<58}|\n")
        file.write(f"|{'Grand Total:':<58} {(total_amount+vat):<58}|\n")
        file.write("+" + "-" * 117 + "+")
    print(f"Invoice created successfully")
except Exception as e:
    print(f"An error occurred while creating the invoice: {e}")
```

10.1.3 Operation.py

```
import write


def display_products(products):
    """
    Display all products in a formatted table.

    Args:
        products (list): List of product data to display
    """
    print("+"+"-" * 100 + "+")
    print(f"|{'ID':<5}| {'Name':<30}| {'Brand':<20}| {'Price':<10}| {'Quantity':<10}| {'Country':<15}|")
    print("+"+"-" * 100 + "+")

    for product in products:
        id = product[0]
        name = product[1][:29] if len(product[1]) > 29 else product[1]
        brand = product[2][:19] if len(product[2]) > 19 else product[2]
        price = str(int(product[3])*2)
        quantity = str(int(int(product[4])*(3/4)))
        country = product[5][:14] if len(product[5]) > 14 else product[5]
```

```
        print(f"|{id:<5}| {name:<30}| {brand:<20}| {price:<10}| {quantity:<10}| {country:<15}|")
    print("+" + "-" * 100 + "+")


def add_summary(shopping_cart):
    """
    Display a summary of products added to the cart.


    Args:
        shopping_cart (list): List of products in the cart
    """
    print("+" + "-" * 100 + "+")
    print(f"|{'Checkout Summary':^100}|")
    print("+" + "-" * 100 + "+")
    print(f"|{'Name of Product':<50}| {'Price':<22}| {'Quantity':<24}| ")
    print("+" + "-" * 100 + "+")
    total=0
    for cart in shopping_cart:
        name = cart[1][:49] if len(cart[1]) > 49 else cart[1]
        price = str(int(cart[3]))
        quantity = cart[4]
        print(f"|{name:<50}| {price:<22}| {quantity:<24}|")
        total += int(cart[3])*int(cart[4])
```

```
    print("+" + "-" * 100 + "+")
    print(f"|{'Total Amount:':<50} {total:<49}|")
    print("+" + "-" * 100 + "+")

def sell_summary(shopping_cart, free):
    """
    Display a summary of products sold with free items.

    Args:
        shopping_cart (list): List of products sold
        free (list): List of free items per product
    """
    print("+" + "-" * 100 + "+")
    print(f"|{'Checkout Summary':^100}|")
    print("+" + "-" * 100 + "+")
    print(f"|{'Name of Product':<50}| {'Price':<7}| {'Quantity':<10}| {'Free Item':<10}| {'Total Quantity':<15}|")
    print("+" + "-" * 100 + "+")
    total=0

    for i,cart in enumerate(shopping_cart):
        free_item=free[i]
        name = cart[1][:49] if len(cart[1]) > 49 else cart[1]
```

```
        price = str(int(cart[3]))
        quantity = cart[4]
        print(f"|{name:<50}|          {price:<7}|          {(int(quantity)-free_item):<10}|
{free_item:<10}| {int(quantity):<15}|")
        total += int(cart[3])*int(int(cart[4])-free_item)
    print("+" + "-" * 100 + "+")
    print(f"|{'Total Amount:':<59} {total:<40}|")
    print("+" + "-" * 100 + "+")


def add_product(products):
    """
    Add new products or update quantities of existing products.

    This function allows:
    - Adding quantity to existing products
    - Creating new products with details
    - Generating invoices for the transaction

    Args:
        products (list): Current list of products

    Returns:
        list: Updated list of products
```

```
    """
    shopping_cart=[]
    while True:
        exist = False
        # Get and validate product name
        while True:
            try:
                name = str(input("Enter product name: "))
                name = name.upper()
                if len(name) == 0:
                    raise ValueError("Product name cannot be empty.")
                break
            except ValueError as e:
                if "Product name cannot be empty." in str(e):
                    print(f"Invalid input: {e}")
                else:
                    print("Invalid input. Please enter a valid product name.")


        # Check if product exists and update quantity
        for product in products:
            if product[1].strip() == name.strip():
                while True:
                    try:
```

```
                quantity = int(input("Enter product quantity to add: "))
                if int(quantity) <= 0:
                    raise ValueError("Enter positive integer.")
                elif int(quantity) > 0:
                    quantity = str(int(quantity))
                    break
            except ValueError as e:
                if "Enter positive integer." in str(e):
                    print(f"Invalid input: {e}")
                else:
                    print("Invalid input. Please enter a valid integer.")

        product[4] = str(int(product[4]) + int(quantity))
        exist = True
        cart=[product[0], product[1], product[2], product[3], quantity, product[5]]
        shopping_cart.append(cart)
        break

# If product doesn't exist, get details and add new product
if exist == False:
    id = str(len(products)+1)
    while True:
        try:
```

```
            brand = str(input("Enter product brand: "))
            brand = brand.upper()
            if len(brand) == 0:
                raise ValueError("Brand name cannot be empty.")
            break
        except ValueError as e:
            if "Brand name cannot be empty." in str(e):
                print(f"Invalid input: {e}")
            else:
                print("Invalid input. Please enter a valid brand name.")


    while True:
        try:
            price = int(input("Enter product price: "))
            if int(price) < 0:
                raise ValueError("Enter positive integer.")
            elif int(price) > 0:
                price = str(int(price))
                break
        except ValueError as e:
            if "Enter positive integer." in str(e):
                print(f"Invalid input: {e}")
            else:
```

```
            print("Invalid input. Please enter a valid integer.")


while True:
    try:
        quantity = int(input("Enter product quantity: "))
        if int(quantity) < 0:
            raise("Enter positive integer.")
        elif int(quantity) > 0:
            quantity = str(int(quantity))
            break
    except ValueError as e:
        if "Enter positive integer." in str(e):
            print(f"Invalid input: {e}")
        else:
            print("Invalid input. Please enter a valid integer.")


while True:
    try:
        country = str(input("Enter product country: "))
        country = country.upper()
        if len(country) == 0:
            raise ValueError("Country name cannot be empty.")
        for char in country:
```

```
                    if not(char.isalpha() or char.isspace()):
                        raise ValueError("Enter valid country name.")
                break
            except ValueError as e:
                if "Country name cannot be empty." in str(e):
                    print(f"Invalid input: {e}")
                elif "Enter valid country name." in str(e):
                    print(f"Invalid input: {e}")
                else:
                    print("Invalid input. Please enter a valid country name.")
        products.append([id, name, brand, price, quantity, country])
        cart=[id, name, brand, price, quantity, country]
        shopping_cart.append(cart)

    # Ask if user wants to add another product
    while True:
        continue_shopping = input("Do you want to add another product? (Y/N): ").strip().upper()
        if continue_shopping == 'N':
            break
        elif continue_shopping == 'Y':
            break
        else:
```

```
            print("Invalid choice. Please enter Y or N.")


    if continue_shopping == 'N':
        break


write.save_products(products)
add_summary(shopping_cart)


# Get and validate vendor name
while True:
    try:
        vendor = str(input("Enter the name of vendor: "))
        vendor = vendor.upper()
        if len(vendor) == 0:
            raise ValueError("Vendor name cannot be empty.")
        for char in vendor:
            if not(char.isalpha() or char.isspace()):
                raise ValueError("Enter valid vendor name.")
        break
    except ValueError as e:
        if "Enter valid vendor name." in str(e):
            print(f"Invalid input: {e}")
        elif "Vendor name cannot be empty." in str(e):
```

```
            print(f"Invalid input: {e}")
        else:
            print("Invalid input. Please enter a valid vendor name.")

    write.add_invoices(shopping_cart,vendor)
    print("Product added successfully.")
    return products

def sell_product(products):
    """
    Sell products with buy 3 get 1 free promotion.

    This function handles:
    - Product selection by ID
    - Quantity validation and stock checking
    - Free item calculation (buy 3 get 1 free)
    - Multiple product purchase in one transaction
    - Invoice generation

    Args:
        products (list): Current list of products

    Returns:
```

```
    list: Updated list of products after sales
    """
    shopping_cart=[]
    free=[]
    display_products(products)


    while True:
        product_id = input("Enter the product ID to buy: ").strip()
        exist = False
        quantitynotFound = False
        # Find product by ID and handle purchase
        for product in products:
            if product[0] == product_id:
                exist = True
                while True:
                    try:
                        quantity = int(input("Enter quantity to buy: ").strip())
                        free_item=int(quantity)//3
                        available_quantity=int(int(product[4])*(3/4))
                        if int(quantity) <= 0:
                            print("Invalid quantity. Please enter a positive number.")
                        elif available_quantity == 0:
                            quantity = 0
```

```
                    free_item = 0
                    print("Product is out of stock.")
                    quantitynotFound = True
                    break
                elif available_quantity < quantity:
                    print(f"Insufficient stock. Available quantity is {available_quantity}.")
                elif available_quantity >= quantity:
                    break
            except ValueError:
                print("Invalid input. Please enter a valid integer.")
        # Update inventory and add to cart
        product[4] = str(int(product[4]) - int(quantity+free_item))
        cart=[product[0], product[1], product[2], int(product[3])*2, str(int(quantity)+free_item), product[5]]
        free.append(free_item)
        shopping_cart.append(cart)
        break

    if exist == False:
        print("Product not found. Select a valid product ID.")
        continue
```

```
        # Ask if user wants to buy another product
        if quantitynotFound == True or quantitynotFound == False:
            while True:
                continue_shopping = input("Do you want to buy another product? (Y/N): ").strip().upper()
                if continue_shopping == 'N':
                    break
                elif continue_shopping == 'Y':
                    break
                else:
                    print("Invalid choice. Please enter Y or N.")

            if continue_shopping == 'N':
                break

    i = 0
    while i < len(shopping_cart):
        if shopping_cart[i][4] == '0':
            shopping_cart.pop(i)
            free.pop(i)
            # Don't increment i since the next item has shifted to this position
        else:
            i += 1  # Move to next item only if we didn't remove anything
```

```
    if len(shopping_cart) == 0:
        print("no product bought.")
        return products

    elif len(shopping_cart) >= 1:
        # Save changes and generate invoice
        write.save_products(products)
        sell_summary(shopping_cart,free)

        # Get and validate customer name - must contain valid characters only
        while True:
            try:
                customer = str(input("Enter the name of customer: "))
                customer = customer.upper()
                if len(customer) == 0:
                    raise ValueError("Customer name cannot be empty.")
                for char in customer:
                    if not(char.isalpha() or char.isspace()):
                        raise ValueError("Enter valid customer name.")
                break
            except ValueError as e:
                if "Enter valid customer name." in str(e):
```

```
            print(f"Invalid input: {e}")
        else:
            print("Invalid input. Please enter a valid customer name.")

    # Generate invoice and confirm purchase
    write.sell_invoices(shopping_cart,customer,free)
    print("Product sold successfully.")
    return products
```

10.1.4 Main.py

```
import read
import write
import operation

def menu():
    """
    Display menu options and get user selection.

    Returns:
        int: User's choice (1-4)
    """
```

```
    print(f"+------------------------------+\n|{'1. Display Products':<30}|\n|{'2. Add Product':<30}|\n|{'3. Sell Product':<30}|\n|{'4. Exit':<30}|\n+------------------------------+")
    try:
        choice = int(input("\nEnter the number that you want to choose: "))
    except ValueError:
        print("Invalid input. Please enter a number.")
        return menu()
    return choice


def main():
    """
    Main program function that runs the WeCare inventory system.

    Loads product data, displays menu options, and processes user choices
    until the user chooses to exit.

    Returns:
        None
    """
    print(f"+------------------------------+\n|{'WeCare':<30}|")
    products = read.load_products()
```

```
    while True:
        choice = menu()


        if choice == 1:
            operation.display_products(products)
        elif choice == 2:
            print(f"\n+------------------------+\n|{'Add New Product':<25}|\n+-----------------
--------+\n")
            products = operation.add_product(products)
        elif choice == 3:
            print(f"\n+------------------------+\n|{'Sell  Product':<25}|\n+----------------------
-+")
            products = operation.sell_product(products)
        elif choice == 4:
            print("\nExited the Program!\n")
            break
        else:
            print("\nInvalid choice. Please try again.\n")


main()
```